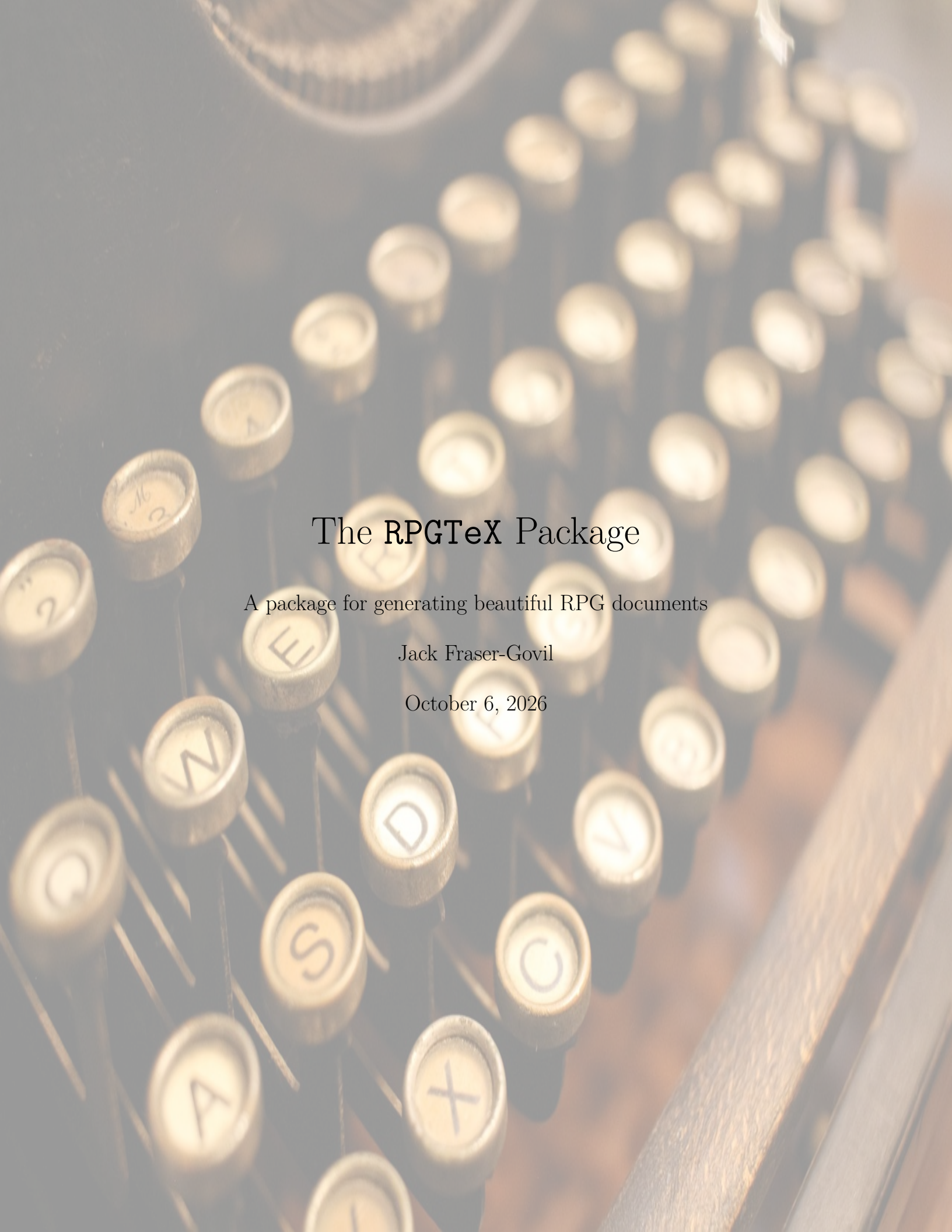




The **RPGTeX** Package

A package for generating beautiful RPG documents

Jack Fraser-Govil

October 6, 2026

Welcome to the *RPGTeX* package. This $\text{\LaTeX}$ package is designed to allow users to flexibly typeset documents associated with Role Playing Games such as *Dungeons & Dragons* -- and many more besides. This package defines a central engine: `rpgcore` which define a number of useful functions and classes, and a flexible set of `themes` which control how those commands are rendered in the final document.

Getting Started

This document is the full reference manual. For a quick example of how to get started with your first *RPGTeX* document, see the [example documents](#), which demonstrate the basic features of this package in a readable fashion.

Attribution & License

This package would not have been possible without the team who developed [its predecessor, the 'DND 5e LaTeX Template'](#). That code was released under an MIT license, the text of which can be found in the `LICENSE` file. `rpgtex` is released under the same license.

AI Usage Declaration

The author of this package used a handful of AI chat tools over the course of the development to help guide the thoughts of the human author. No automated coding tools were used anywhere in this package, and all code was written by hand. All mistakes and bad decisions are organic in nature.

Contents

1: Installation & Usage	
Getting R _P G _T eX	1
Configuring R _P G _T eX.....	2
Package & Class Usage	2
Compiling rpgtex Documents	3

Part I: rpgtex For Users

2: Options & Configuration	
Package Options	6
Colo(u)rs.....	8
Command Line Interface.....	9
3: Commands & Macros	
Themes & Colours	10
Document Structure.....	10
Text Utilities.....	13

Full-Page Images.....	15
Switches	15

4: Environments	
Callouts: RpgNarration, Sidebar & Tip.....	17
RpgTable.....	18
Abstracts.....	19
Filigree	19
Secrets.....	20
Maps	21
Cards	22
Rule Environments.....	24

Part II: rpgtex For Designers

5: Fonts	
Font Commands.....	26
Font Elements	27

Chapter 1: Installation & Usage

Getting R_PG_TE_X

There are a number of different ways to acquire R_PG_TE_X. Once you have installed it, it is vital to ensure that it is properly configured (see below).

Direct Installation

The simplest way to use R_PG_TE_X is to install it on the `texmf` path, where the compiler can automatically find it:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex" %make sure the directory exists
git clone https://github.com/DrFraserGovil/rpgtex.git "$(kpsewhich -var-value
TEXMFHOME)/tex/latex/rpgtex"
```

This will clone the repository into your L^AT_EX path.

Indirect Installation

If you want to tinker with R_PG_TE_X -- such as by creating a new theme -- it is helpful to have it in a more accessible location. Clone the repository into a location of your choice:

bash

```
git clone https://github.com/DrFraserGovil/rpgtex.git /your/rpgtex/directory
```

You then have two options to make the package visible to the compiler:

Use TEXINPUTS

Setting the environment variable `TEXINPUTS` allows the compiler access:

bash

```
TEXINPUTS=~ /your/rpgtex/directory/::
```

(Or similar commands, depending on your shell).

Use Symlinks

You can symlink the install location to the `texmf` directory, allowing the compiler to act as if you had performed the `texmf` installation:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex"
ln -sf ~/your/rpgtex/directory "$(kpsewhich -var-value TEXMFHOME)/tex/latex/rpgtex"
```

Overleaf (Not recommended!)

We do not recommend using Overleaf since the free-tier subscription has reduced compilation times drastically, making compiling documents using complex packages such as this one extremely difficult. Nevertheless:

1. Download this GitHub repository as a ZIP archive using the Clone or download link above.

2. On Overleaf, click the New Project button and select Upload Project. Upload the ZIP archive you downloaded from this repository.
3. Manually create the file `rpg-config.cfg` with the contents `\edef\RpgPackagePath{}`. This replaces the configuration step described below.

Configuring RPGTeX

Because R_PG_TE_X comes packaged with some resources (namely some fonts) for which T_EX does not support relative-pathfinding, it is necessary to provide R_PG_TE_X with a value for the `\RpgPackagePath`.

`\RpgPackagePath` (Variable)

A variable holding the absolute path to the root directory of the rpgtex install location.

We provide a number of different ways to set this variable:

1. The provided `rpglatex` compiler automatically inserts a definition based on a `kpsewhich`, using the Command Line Interface (page 9).
2. If the variable does not exist, the R_PG_TE_X engine itself tries the following
 - (a) Searches for the file `core/rpg-config.cfg`, which (hopefully) contains a definition¹, and `\inputs` it.
 - (b) If no file is found, R_PG_TE_X used the `\write18` utility to dispatch its own calls to `\kpsewhich` and build a config file in the expected location. This requires the compiler be run with `-shell-escape` active
 - (c) If this doesn't work, the compiler exits, and suggests that the user manually creates the config file:

```
cd <rpgtex root directory>
cmd="\edef\RpgPackagePath{ $(pwd) }"
echo $cmd >> core/rpg-config.cfg
```

3. We provide a shell script which executes these commands for the user.

The extra configuration steps need only be run once -- once the config file exists, the package will be able to configure itself.

Why is configuration necessary?

T_EX is generally set up so that when a file calls `include` or `input` it is possible to use filepaths relative to the package itself. `package.sty` can call `\input{dir/subpackage.tex}` and it will know to first check for the file relative to `package.sty`; even if the package resides within the texmf path and the user has no idea where `package.sty` is.

An annoying exception to this is fonts and typefaces. `xelatex` searches for fonts based on *filepaths relative to the current working directory* -- or from those installed in as system fonts.

Since R_PG_TE_X includes several (license free) typefaces as part of the provided themes, this poses a problem. We must either require that:

1. R_PG_TE_X documents can only be prepared in restricted locations relative to the install location of R_PG_TE_X.
2. Users must identify and specify the R_PG_TE_X root path when preparing a document
3. Users must install the provided fonts to the system path
4. R_PG_TE_X must be configured to know 'where it is', and so provide an absolute filepath to the internal fonts.

The Configuration step is the most portable and easiest-to-use of these options.

Package & Class Usage

R_PG_TE_X can be used either as a standalone package, or as part of a number of classes

¹In the form of `\edef\RpgPackagePath{ path/to/package }`

Standalone Package

The standalone package can be used directly by including the `RPGTeX` package:

```
\documentclass{arbitrary-class}
\usepackage[options]{rpgtex}
```

This will load only the core commands into the document, and (unless called explicitly) no themes will be imported. Using the package in this way does not activate any of the commands which change the overall geometry, background or headers of the document.

Classes

RPGTeX can also be loaded through a number of classes which drastically alter the appearance of the document, defining new geometries backgrounds and adding headers.

The provided classes are:

1. `rpgbook` (page ??). Based on the standard book class, this is designed for larger RPG documents.
2. `rpghandout` (page ??). Based on the article class, this is designed for shorter documents
3. `rpgcard` (page ??). A small-document class designed for creating modular 'handout' cards for items, spells or abilities.

Compiling rpgtex Documents

RPGTeX uses the `fontspec` package to allow custom fonts, and therefore requires compiling with `xelatex` or `luatex`:

```
xelatex main.tex #works
luatex main.tex #works
pdflatex main.tex #fails if fonts option passed
```

So long as RPGTeX is on the user's latex path, and the package properly configured (page 2) no further compilation steps are required. However, for ease of use, we provide the `rpglatex` compiler as part of the RPGTeX distribution.

The rpglatex compiler

RPGTeX is shipped with a special compiler, `rpglatex`. This is simply a python3 script which acts as a wrapper around either `xelatex` or `luatex`, but includes several quality-of-life changes to the interface to make it easier to use with `rpgtex`.

```
rpglatex [options] <file>
```

`rpglatex` has the following features:

Feature	Description	Options
Compiler Selection	The <code>xelatex</code> compiler is selected by default, but the <code>-l</code> , <code>-luatex</code> flags set it to use <code>luatex</code> instead.	<code>-l</code> , <code>-luatex</code>
Build Directory	Compilation files (<code>.aux</code> , <code>.log</code> etc.) are stored in a build directory. The default is <code>.build</code> in the calling location, but can be changed with the <code>-b</code> flag	<code>-b <build dir></code>
Output Naming	The name of the output file can be changed from the default (equal to the input tex name)	<code>-o <output name></code>
Multi-pass Compiling	By default, the compiler runs twice in a row to enable references and <code>tikz[remember]</code> commands to function. A full three-compilation suite (necessary for very complex or reference-heavy documents) can be activated with the <code>-f</code> , <code>-full</code> flag	<code>-f</code> , <code>-full</code>

Volume Control	latex is notoriously noisy, producing copious output. By default, this is suppressed and only a summary is printed. The summary can be removed (rendering it completely silent) with the <code>-q</code> command, or the original output recovered in verbose mode; <code>-v</code>. These outputs are always overridden if a compilation error occurs, in which case the full trace is output to the console.	<code>-q, -v</code>
Auto-bibtex	If the <code>-r</code> or <code>-ref</code> flag is set, <code>bibtex</code> is automatically called in between the multi-compilation steps	<code>-r, -ref</code>
Auto-visualisation	If the <code>-show 1</code> option is set (which it is by default), the compiler will attempt to open the output file upon completion of the compilation; automatically opening or context-switching to the document. This can be turned off by calling <code>-show 0</code>	<code>-show</code>
Print Mode	A special interface for <code>RPGTeX</code> , this uses the <code>\RpgCMD</code> interface (page 9) to inject code into the latex document, setting the <code>bg=print</code> mode and suppressing the background output.	<code>-p, -print</code>

PART I

rpgtex For Users

Chapter 2: Options & Configuration

This chapter describes the ways in which the behaviour of RPGTeX may be configured without writing any new code: the options which may be passed to the package or class, the named colours used by its elements, and the command line interface for changing these options at compile time.

Package Options

Whether RPGTeX is loaded directly as a package or indirectly through one of its classes, options may be passed to it which change the behaviour of the resulting document:

```
\documentclass[<options>]{rpgbook}  
\usepackage[<options>]{rpgtex}
```

Each option takes one of two forms:

Key-value options: The name of the option is followed by an equals sign and a value:

```
\usepackage[bg = print, theme = dnd]{rpgtex}
```

Flags: Only the name of the option is provided. Its presence enables the associated behaviour:

```
\usepackage[justified]{rpgtex}
```

The available options are:

background <flag>

If present, activates the 'paper background' modules, which control the paper colour and background images, the footer decorations, and the spacing around floats.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

bg = full / print / none

Controls the behaviour of the background and footer decorations. The value must be one of the three listed above, else an error is thrown. The default value is **full**.

The most visible effect of this option is to control whether the 'background paper' set by `\RpgThemePaperImage` (page ??) and `\RpgThemePaperColor` (page ??), and the footer decorations set by `\RpgThemeFooterDecoration` (page ??), appear in the document:

Value	Paper	Footer Decorations
full	✓	✓
print	✗	✓
none	✗	✗

This option also changes the behaviour of the `rpgcard` class (page ??), and other environments may similarly alter their colours or layouts in response to its value.

columns = 1 / 2

Places the document into either `onecolumn` or `twocolumn` mode. The default value is 2.

More advanced column layouts require the user to manually use the `multicols` environment.

This option only has an effect when using one of the `RPGTeX` classes.

`font` <flag>

If present, activates the font modules, applying the package fonts wherever possible.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`justified` <flag>

If present, sets the text to be justified throughout the document. By default, text is left-aligned ('ragged right').

`noemph` <flag>

If present, the default behaviour of `\emph` is left unchanged. By default, `\emph` is redefined to be an alias of `\RpgKey` (page 14).

`nomultitoc` <flag>

If present, disables the multi-column table of contents.

`oneside` <flag>

If present, places the document into one-sided mode. This disables alternating footers, but allows for consistent asymmetric margins (as in this document, where the left margin is enlarged to accommodate macro names).

This option only has an effect when using one of the `RPGTeX` classes.

`openright` <flag>

If present, places the document into right-opening mode, such that chapters always begin on a right-hand page. By default, chapters may begin on any page (see the documentation of the `book` class for more details).

This option only has an effect when using one of the `RPGTeX` classes.

`sections` <flag>

If present, activates the sections modules, including the part-page overloads and any theme behaviour which modifies the section commands.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`size` = <font-size>pt

Sets the base font size of the document, if permitted by the parent class. The default value is 10pt.

This option only has an effect when using one of the `RPGTeX` classes.

The allowed values depend on the class being used (see the class section, beginning on page ??).

`theme` = <theme name>

The initial value passed to `\RpgSetTheme` (page 10) once package initialisation is complete.

The default value is `default`, which activates the Default Theme (page ??).

`themepath = </path/to/themes/>`

Calls `\RpgSetThemePath` (page 10) before the theme is loaded, allowing custom themes to be used.

The default value is `themes`, which `TEX` locates within the `RPGTeX` installation. A theme called `name` is then expected to be stored in `<themepath>/name/name.cfg`.

`title <flag>`

If present, activates the titling module and any theme presets which alter the behaviour of `\maketitle`.

This flag is set by default when loading a class, but must be passed explicitly to enable this module when loading the package directly.

Colo(u)rs

`RPGTeX` defines a number of named colours¹ which are used by its various elements. The default values listed below may be overridden by the active theme.

Co-varying Colours

The following colours are coupled together, and by default all take the same value:

ThemeColor A general-purpose colour from which the other co-varying colours inherit. Default: `white!60!black`.

SidebarColor The background colour of the `RpgSidebar` (page 17) environment.

TableColor The background colour of alternate rows in an `RpgTable` (page 18).

TipColor The background colour of the `RpgTip` (page 17) environment.

Calling `\RpgSetThemeColor` (page 10) assigns a new value to **ThemeColor** and all three co-varying colours simultaneously:

```
\RpgSetThemeColor{blue!20!white}
```

Independent Colours

The remaining colours are set individually:

ContourInnerColor The default colour of the inner text produced by `\RpgContour` (page 13). Default: `black`.

ContourOuterColor The default colour of the outer contour drawn around text by `\RpgContour` (page 13). Default: `white!50!black`.

FiligreeBackColor The background colour of the `RpgFiligree` (page 19) environment. Default: `white`.

FiligreeColor The colour used to draw the filigree frame applied by the `rpgfiligree` style. Default: `black`.

NarrationColor The background colour of the `RpgNarration` (page 17) environment. Default: `F7F2E5` (HTML).

PaperColor The background colour of each page, applied only when the paper background is enabled (see the `bg` option, page 6). Default: `white`.

SecretColor The default frame colour of the `RpgSecret` (page 20) environment. Default: `black!30!white`.

Any of these colours, including the co-varying colours, may be modified individually using the standard `xcolor` interface:

```
\definecolor{NarrationColor}{HTML}{FFFFFF}  
\colorlet{SidebarColor}{red!10!white}
```

Modifying a co-varying colour in this way changes only that colour, leaving the others unaffected.

¹Although this document uses British spelling, the colour names follow the American spelling used throughout `LATEX` and `xcolor`, and must be written exactly as given (including capitalisation).

Command Line Interface

L^AT_EX does not natively provide a 'command line interface' through which a document may be modified at compile time: any change must be made within the source file itself. It is, however, often convenient to compile the same document in several configurations without editing it -- most notably to produce both a standard version and a printer-friendly version (using `bg = print`, which removes the paper background to reduce ink usage).

To support this, R^PG_TE_X checks for a pseudo command-line variable when processing its options. Because T_EX treats the text passed to it on the command line as the beginning of its input stream, this variable can be defined before the document itself is read.

`\RpgCMD` (Variable)

If defined, holds a comma-separated list of package options (page 6), which are applied after all other options have been processed. The options are applied in the following order, with later values overriding earlier ones:

1. The defaults set by the class (if one is being used).
2. The options passed in the document, via `\documentclass` or `\usepackage`.
3. The contents of `\RpgCMD`.

Since the options are processed before they are passed to the parent class, options which affect the class (such as `columns` or `size`) may also be set in this way.

`\RpgCMD` is defined by passing a short piece of T_EX code to the compiler before the document is input:

```
xelatex "\def\RpgCMD{<rpg-options>} \input{<document>}"
```

This compiles `<document>` as though `<rpg-options>` had been passed to the class or package. For example, the following produces a printer-friendly, single-column version of a document without modifying the source:

```
xelatex "\def\RpgCMD{bg=print, columns=1} \input{document.tex}"
```

The `rpGLaTeX` compiler (page 3) performs this insertion automatically: the `-p` (`--print`) flag sets `bg=print`, allowing the user to switch between the standard and print versions of a document with a single flag:

```
rpGLaTeX document.tex -p
```

Chapter 3: Commands & Macros

Themes & Colours

`\RpgSetTheme { name }`

Loads the theme called **name**, replacing the appearance of any previously loaded theme. The theme is first searched for at `<themepath>/<name>/<name>.cfg`; if no such file exists, **name** is instead treated as a direct path to a theme file. An error is raised if neither file exists.

The initial theme is set by the **theme** option (page 6). `\RpgSetTheme` may be called at any point in the document, but since its effects are immediate, it is best called at the start of a new page.

`\RpgSetThemeColor { colour }`

Sets **ThemeColor** and all of the co-varying colours to **colour** (page 8).

`\RpgSetThemePath { path }`

Sets the directory searched by `\RpgSetTheme`, allowing custom themes to be stored outside of the **RPGTeX** installation. The path is not checked until a theme is loaded. The initial value is set by the **themepath** option (page 8).

Document Structure

RPGTeX extends the standard **L^AT_EX** commands for structuring a document. It provides title and part pages designed by the active theme, records the names of the current headings so that they may be reused elsewhere, and allows ‘fake’ headings to be added to the table of contents.

The Title & Sections Options

Much of the behaviour described in this section is provided by redefining standard **L^AT_EX** commands, and is controlled by two package options (page 6). Both are active by default when using one of the **RPGTeX** classes; otherwise, they must be passed to the package explicitly.

title Redefines `\maketitle` to produce the title designed by the active theme (equivalent to calling `\RpgMakeTitle`). Otherwise, `\maketitle` retains its original **L^AT_EX** behaviour.

sections Activates *sections mode*, which:

- Redefines `\chaptermark`, `\sectionmark`, `\subsectionmark` and `\subsubsectionmark` so that, in addition to their usual behaviour, they store the name they are given (page 12).
- Sets `\secnumdepth` to 0, so that sections and their subdivisions are unnumbered.
- Replaces `\part` with a version which records the part name and draws a full-page part page using the active theme. The starred form, `\part*`, records the name but otherwise falls back to the original **L^AT_EX** command.

Themes may additionally alter the formatting of the sectioning commands when sections mode is active.

All of the other commands in this section, including `\RpgMakeTitle`, are available regardless of these options, although the stored heading names are only updated in sections mode.

Title Pages

The title page draws on the usual `\title` and `\author` commands, along with the following additions.

`\cover { path/to/image }`

Sets an image to be used as the full-page background of the cover page created by `\maketitle` in `cover` mode (see below). The theme's title design is drawn over the top of the image.

The default value is empty, in which case the title design is drawn on a plain page.

`\maketitle` `{}`

`\RpgMakeTitle` Creates the title of the document using the design provided by the theme (only when the `title` option is active; see above). There are two title modes:

cover A full-page cover, drawn over the `\cover` image (if one has been set). This is the default for the `rpgbook` class.

header A title heading placed at the top of the first page of text.

The mode is selected with `\RpgThemeTitleMode{ cover / header }`. If the theme does not define a title for the active mode, the original L^AT_EX `\maketitle` is used instead.

`\RpgMakeTitle` is equivalent to `\maketitle`, but remains available even if the `title` option is not active, or if another package redefines `\maketitle`.

`\RpgCoverTransparency` `{ value }`

Fades the `\cover` image towards white, which can help to keep the title legible over a busy image. The `value` is the opacity of a white layer placed over the image, between 0 (the image is unaltered) and 1 (the image is completely hidden). The default value is 0.

`\RpgGetAuthor` *(Variable)*

`\RpgGetAuthor*` The author set by `\author`, formatted with the `\RpgFontSubTitle` (page 27) font element. The starred form gives the author as plain text.

`\RpgGetSubtitle` *(Variable)*

`\RpgGetSubtitle*` The subtitle set by `\subtitle`, formatted with the `\RpgFontSubTitle` (page 27) font element. The starred form gives the subtitle as plain text.

`\RpgGetTitle` *(Variable)*

`\RpgGetTitle*` The title set by `\title`, formatted with the `\RpgFontTitle` (page 27) font element (page 26). The starred form gives the title as plain text, without applying the font.

`\subtitle` `{ text }`

Sets the subtitle of the document. As with `\title` and `\author`, this has no visible effect by itself: the subtitle is displayed by `\maketitle` if the theme includes it. The default value is empty.

Part Pages

`\part` *(*)* `[<image>]{ name }`

In sections mode (page 10), `\part` is redefined to create a full-page part page using the design provided by the theme. Its behaviour depends on the form used:

`\part{name}` Adds the part to the table of contents and creates a part page on a plain background, or over the default part image if one has been set with `\RpgDefaultPartImage`.

`\part[image]{name}` As above, but uses `image` as the full-page background.

`\part*{name}` Uses the original `\part` command of the underlying class. Any image argument is ignored.

In all cases, `name` is recorded in `\RpgPartName` (page 12).

`\RpgDefaultPartImage` `{ path/to/image }`

Sets an image to be used as the full-page background of every part page which is not given its own image. By default, no image is used.

`\RpgPartTransparency { value }`

Fades all subsequent images set by `\part` image towards white, which can help to keep the text legible over a busy image. The `value` is the opacity of a white layer placed over the image, between 0 (the image is unaltered) and 1 (the image is completely hidden). The default value is 0.

Section Names

Many RPG documents display the name of the current chapter or section outside of the heading itself -- in a running header, a footer, or a decorative page border, for example. To support this, `RPGTeX` records the names of the current part, chapter, section, subsection and subsubsection, so that they may be reused anywhere in the document.

These names are only recorded in sections mode (page 10).

Each of the following variables is initially blank, and holds its value until the next command of the same level is called: beginning a new chapter does not, for example, reset `\RpgSectionName`.

`\RpgChapterName` *(Variable)*

The name given to the last `\chapter` command, or set by calling `\chaptermark`. Starred chapters (`\chapter*`) do not update this value.

`\RpgPartName` *(Variable)*

The name given to the last `\part` or `\part*` command.

`\RpgSectionName` *(Variable)*

The name given to the last `\section` command, or set by calling `\sectionmark`. Starred sections (`\section*`) do not update this value.

`\RpgSubsectionName` *(Variable)*

The name given to the last `\subsection` command, or set by calling `\subsectionmark`. Starred subsections (`\subsection*`) do not update this value.

`\RpgSubsubsectionName` *(Variable)*

The name given to the last `\subsubsection` command, or set by calling `\subsubsectionmark`. Starred subsubsections (`\subsubsection*`) do not update this value.

`\RpgGetName { part / chapter / section / subsection / subsubsection }`

Returns the name stored for the given level (equivalent to the corresponding variable above). Any other input results in an error.

Since these names (other than that of the part) are taken from the `mark` commands, they may be set manually, without creating a new heading, by calling the relevant `mark` command directly.

```
We are in Chapter '\RpgGetName{chapter}',  
section '\RpgGetName{section}'.
```

```
\subsubsection{An example subsubsection}
```

```
Now I am in  
'\RpgGetName{subsubsection}'.
```

We are in Chapter `Commands & Macros',
section `Document Structure'.

An example subsubsection

Now I am in `An example subsubsection'.

Fake Sections

It can be helpful to be able to integrate common RPG elements (such as full page maps or images) into the document structure. The following command provides a simple interface for spoofing in a section (or part, chapter etc.) into the table of contents, without actually invoking a sectioning command:

`\RpgFake { level }{ name }`

Mimics the creation of a new heading at the given `level` without printing a heading in the document. Specifically, it:

- Adds an entry called `name` to the table of contents at the given level, with a hyperlink anchor at the current position.
- Updates the stored name for that level, as if a real heading had been created. For `chapter` to `subsubsection` this is done by calling the corresponding `mark` command, so any running headers are also updated.

The `level` must be one of `part`, `chapter`, `section`, `subsection` or `subsubsection`. Any other value results in an error.

```
\subsubsection{Real Subsection}
I believe I am in
  '\RpgGetName{subsubsection}'.

\RpgFake{subsubsection}{Fake Subsection}
Now I believe I am in
  '\RpgGetName{subsubsection}'
```

Real Subsection

I believe I am in `Real Subsection'.
Now I believe I am in `Fake Subsection'

Text Utilities

RPGTeX provides a number of commands for applying decorative styles to text and transforming it in ways useful for writing in an RPG context.

Any fonts or styles they apply are those assigned by the theme (page 26).

`\RpgContour [<options>]{ text }`

Typesets `text` with a coloured outline (contour), using the `contour` package. The optional argument accepts the following keys:

inner The colour of the text itself. Default: `ContourInnerColor` (page 8).

outer The colour of the outline. Default: `ContourOuterColor`.

style Formatting commands (such as `\bfseries` or `\Huge`) applied to the text.

Contours cannot automatically linebreak, instead the text is split into lines at each `\\`, and each word is contoured individually so that spacing is preserved. As a consequence, formatting commands placed within `text` may only affect the first word: all formatting should therefore be passed through the `style` key.

```
\RpgContour[inner=red,
  outer=black]{Contoured text}

\RpgContour[style=\Large\bfseries]{Styled
text \\ over two lines}
```

Contoured text
Styled text
over two lines

`\RpgDice { expression }`

Parses a dice expression of the form `<count>d<sides><modifier>` and typesets it in the format defined by the theme. The expression must satisfy the following rules:

- It contains a dice symbol (**d** or **D**), immediately followed by the number of sides.
- The dice symbol may be preceded by the number of dice. If it is omitted, the theme determines how the expression is displayed.
- Anything following the number of sides is treated as the modifier, which may be any numerical expression (such as **+3*2**). It is evaluated to a single number before being displayed.

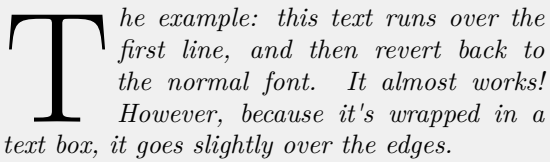
Leading and trailing spaces are ignored. If the expression does not match this form, it is printed unchanged.

<code>\RpgDice{1d6-2}</code>	1d6-2
<code>\RpgDice{2D6 + 3*2^2}</code>	2d6+12
<code>\RpgDice{d8}</code>	d8
<code>\RpgDice{not a dice roll}</code>	not a dice roll

`\RpgDropCap` [**<options>**]{ **letter** }{ **text** }

Begins a paragraph with a large decorative initial (a ‘drop cap’), using the `lettrine` package. The **letter** is typeset in the `\RpgFontDropCap` (page 28) font element, and **text** -- intended to be the remainder of the first line -- in the `\RpgFontDropCapAfter` (page 28) font element.

The drop cap spans a number of lines set by the theme (four, by default), and is separated from the following text by the length `\RpgDropCapSeparation`, which may be changed with `\setlength`. The optional argument accepts any `lettrine` options, which override these defaults.

<code>\RpgDropCap{T}{he example: this text runs over the first line, and then revert back to the normal font. It almost works! However, because it's wrapped in a text box, it goes slightly over the edges.}</code>	
--	---

`\RpgKey` { **text** }

`\emph` Typesets **text** in the `\RpgFontEmphasis` (page 27) font element, giving a consistent appearance to keywords (such as conditions or game terms).

When the `font` option is active, and unless the `noemph` option is passed (page 6), `\emph` is redefined to be an alias of `\RpgKey`. This replaces its usual ‘context-aware’ behaviour (italics within upright text, and vice versa) with the keyword style.

<code>\emph{text}</code>	<i>text</i>
<code>\RpgKey{more text}</code>	more text

`\RpgOrdinal` [**<format>**]{ **number** }

Typesets an integer followed by its English ordinal suffix (**st**, **nd**, **rd** or **th**), correctly handling 11th, 12th and 13th. The optional **format** is a command applied to the suffix only, such as `\textbf`.

<code>\RpgOrdinal{1}, \RpgOrdinal{12}, \RpgOrdinal{23}</code>	1st, 12th, 23rd
<code>\RpgOrdinal[\textbf]{133}</code>	133rd

`\RpgPage` [**<style>**]{ **label** }

Typesets ‘page’ followed by the page number of **label**, optionally enclosed in brackets. The **style** may be one of **t** (no brackets; the default), **p** (parentheses), **b** (square brackets) or **c** (curly braces). Any other value encloses the reference in question marks, to make the error easy to spot.

<code>\RpgPage{S:PackageOptions}</code>	
<code>\RpgPage[p]{S:PackageOptions}</code>	page 6
<code>\RpgPage[b]{S:PackageOptions}</code>	(page 6)
<code>\RpgPage[c]{S:PackageOptions}</code>	[page 6]
	{page 6}

`\RpgPlural (*)[<plural>]{ count }{ word }`

Typesets `count` followed by `word`, adding an `s' to the word unless `count` is 1. Irregular plurals may be given in the optional argument, which replaces the word whenever `count` is not 1. The starred form omits the count, printing only the word.

<code>\RpgPlural{1}{hat}, \RpgPlural{2}{hat}</code>	
<code>\RpgPlural[octopodes]{1}{octopus},</code> <code>\RpgPlural[octopodes]{3}{octopus}</code>	1 hat, 2 hats 1 octopus, 3 octopodes
<code>\RpgPlural*{3}{hat}</code>	hats

`\RpgSuperOrdinal { number }`

Equivalent to `\RpgOrdinal[\textsuperscript]{ number }` (page 14), automatically applying a superscript style command.

<code>\RpgSuperOrdinal{42}</code>	42 nd
-----------------------------------	------------------

Full-Page Images

We use the `eso-pic` library for formatting whole-page images and placing content at specific positions on the page.

`\RpgPut [<width>][<format>](<anchor>){ x }{ y }{ content }`

Places `content` at a fixed position on the page, measured from the bottom-left corner. It may only be used within a page overlay, such as the first argument of `\RpgWholePageImage`.

The content is set in a paragraph of the given `width` (default: `\linewidth`), with `format` applied at the start (default: `\centering`).

The `anchor` determines which point of the content is placed at (`x`, `y`): `c` (centre; the default), `t`, `b`, `l`, `r`, or a combination such as `tl`.

`\RpgWholePageImage [<overlay>]{ path/to/image }[<fade>]`

Places an image so that it fills the whole of a new page, clearing the page on either side. This is useful for maps, handouts and illustrations.

The optional `overlay` is drawn on top of the image, and may contain any number of `\RpgPut` commands. The final optional argument (given in square brackets, after the path) fades the image towards white, as described for `\RpgCoverTransparency` (page 10).

Switches

Switches are named true/false values which can be used to change the content of a document without editing its text. For example, `RPGTeX` uses switches to decide whether secrets are shown, and whether rule environments are displayed as cards. All switches are global: changing a switch affects the whole of the remaining document.

Switches are designed to act as boolean counterparts to traditional $\text{\TeX}$ counters, and so have a similar interface.

The primary use-case for switches in $\text{\RPGTeX}$ is for the 'Cardable Environments', allowing the same document to typeset as both inline text, and as a 'playing card module' depending on the value of the associated switch.

```
\RpgIfSwitch { name }{ if-on }{ if-off }
```

Typesets `if-on` if the switch is on, and `if-off` otherwise.

```
\RpgNewSwitch { name }[<value>]
```

Creates a new switch called `name`, with an initial `value` (default: `false`). An error is raised if the switch already exists.

```
\RpgSwitch { name }{ value }
```

Sets an existing switch to the chosen value. Accepted input strings are:

Boolean State	Input Strings
True	on, On, true, True, 1
False	off, Off, false, False, 0

Any other value raises an error, as does a switch which does not exist.

```
\RpgToggle { name }
```

Turns an existing switch on if it is off, and off if it is on.

<pre>\RpgNewSwitch{HardMode} The dragon has \RpgIfSwitch{HardMode}{500}{200} HP. \RpgToggle{HardMode} The dragon has \RpgIfSwitch{HardMode}{500}{200} HP.</pre>	<pre>The dragon has 200 HP. The dragon has 500 HP.</pre>
--	--

Chapter 4: Environments

RPGTeX provides a number of environments for the elements which appear throughout RPG books: boxed text, tables, map keys, secrets and playing cards. Their appearance (fonts, colours and decoration) is set by the active theme, but may be adjusted through their options.

Many of these environments are built on the `tcolorbox` package. Unless stated otherwise, any option which is not recognised by an environment is passed on to `tcolorbox`, so all of its usual options (such as `colframe` or `breakable`) may also be used.

Callouts: RpgNarration, Sidebar & Tip

Callouts are boxed blocks of text which stand apart from the main text, the precise appearance determined by the theme. There are three types of callout, intended for different purposes:

RpgNarration	Text to be read aloud, such as the description of a scene at the start of an adventure.
RpgSidebar	Important information, such as key rules or summaries, which readers <i>should</i> pay attention to.
RpgTip	Helpful additions, such as tips and trivia, which are not essential but are too large for a footnote.

Each callout accepts the following options, in addition to the usual `tcolorbox` options:

Option	Effect
color	The background colour of the box (and its title). The default varies with the callout type. This is an alias (which overrides) the usual <code>tcb/colback</code> option.
filigree	If present, decorates the box with the theme's filigree frame (page 19).

```
\begin{ RpgNarration } [<options>]
```

A box for text to be read aloud. It has no title, and its text is typeset in the `\RpgFontNarration` (page 28) font element on a `NarrationColor` background.

```
\begin{RpgNarration}[color=blue!30!white]
  This is text that you would read out
  loud to players, describing a scene.
  It will always be blue, even if the
  theme says otherwise -- because the
  optional argument takes priority.
\end{RpgNarration}
```

This is text that you would read out loud to players, describing a scene. It will always be blue, even if the theme says otherwise -- because the optional argument takes priority.

```
\begin{ RpgSidebar } [<options>]{ title }
```

A box for important information, with a title. The title and body are typeset in the `\RpgFontSidebarTitle` (page 28) and `\RpgFontSidebarBody` (page 28) font elements, and the background is `SidebarColor`.

```
\begin{RpgSidebar}{A Sidebar}
  This is an important block of text,
  that you should pay attention to.
\end{RpgSidebar}
```

A Sidebar

This is an important block of text, that you should pay attention to.

```
\begin{ RpgTip } [<options>]{ title }
```

A box for helpful additions, with a title. The title and body are typeset in the `\RpgFontTipTitle` (page 28) and `\RpgFontTipBody` (page 28) font elements, and the background is `TipColor`.

```
\begin{RpgTip}{A Tip}
  This is some helpful - but not vital -
  text.
\end{RpgTip}
```

A Tip

This is some helpful - but not vital - text.

RpgTable

`\RpgRowStyle { format }`

Applies **format** to every cell of the current row of an **RpgTable**. It must be placed at the start of the row. This is the mechanism used to style the header row, and may be used to highlight any other row in the same way.

`\begin{ RpgTable } [<options>]{ columns }`

Typesets a table in the style of the theme. By default this is a **tabularx** table which fills the width of the line, so the usual column types (**l**, **c**, **r**, **p**{<width>}, ...) may be used alongside the stretching **X** column. **RPGTeX** also provides a centred stretching column, **Y**.

By default, the table:

- Typesets its first row as a header, in the **\RpgFontTableHeader** (page 28) font element, and the remaining rows in **\RpgFontTableBody** (page 28).
- Colours alternate rows in **TableColor** (page 8), beginning with the second row.

The following options are available:

Option	Effect	Default
breakable	If present, uses xltabular , allowing the table to break across pages. This is only possible in one-column mode.	off
color	The colour used for the coloured rows.	TableColor (page 8)
color-both	If present, colours every row.	off
color-odd	If present, colours the odd rows (including the header) instead of the even rows.	off
filigree	If present, wraps the table in an RpgFiligree (page 19).	off
filigree-hgap	The horizontal padding between the filigree and the table.	1pt
filigree-opacity	The opacity of the filigree's background.	1
headskip	The vertical space below the title.	5pt plus 2pt minus 2pt
intro	Introductory text, typeset between the title and the table.	(empty)
introskip	The vertical space below the introduction.	5pt plus 2pt minus 2pt
noheader	If present, the first row is typeset in the same style as the rest of the table.	off
simple	If present, uses a plain tabular , whose width is determined by its contents. The width , breakable and noheader options are then ignored, and the first row is not styled as a header.	off
title	A title, typeset above the table in the \RpgFontTableTitle (page 28) font element.	(empty)
vskip	The vertical space above and below the table.	9pt plus 3pt minus 3pt
width	The width of the table.	\linewidth

```

\begin{RpgTable}[width=0.85\linewidth,
color=green!30!white]{lX}
  Header 1 & Header 2
  \\
  Text & Text fills up to 85\% of the
    line width then breaks
  \\
  Alternating & This row is transparent
  \\
  Colour & but this one is green
\end{RpgTable}

%suppress header formatting, add title and
  filigree:
\begin{RpgTable}[title={Test
Table},noheader, filigree]{XlX}
  Plain & Header & Text
  \\
  Now there's no & difference & between
    the header and the main
  \\
  body
\end{RpgTable}

```

Header 1	Header 2
Text	Text fills up to 85% of the line width then breaks
Alternating	This row is transparent
Colour	but this one is green

opacityback=1,left=1pt,right=1pt

Test Table

Plain	Header	Text	
Now there's no	difference	between the header and the main	
body			

Abstracts

```
\begin{ RpgAbstract } [<title>] [<width>] [<gap>]
```

abstract Typesets a short, centred summary block, such as the introduction to an adventure. The **title** (default: ``Abstract'`) is typeset in the `\RpgFontAbstractTitle` (page 28) font element, and the body in `\RpgFontAbstractBody` (page 28). The block has the given **width** (default: `0.7\linewidth`), with a vertical **gap** above and below it (default: 15pt). In two-column mode, the block spans both columns. The standard **abstract** environment is redefined to be identical to **RpgAbstract**.

```
\begin{ summary } [<title>] [<width>] [<gap>]
```

Identical to **RpgAbstract**, but with the default title ``Summary'`.

Filigree

Filigree is a decorative frame, drawn around the outside of a box in place of its usual border. Its design is provided by the theme, and it is drawn in **FiligreeColor** (page 8).

```
\begin{ RpgFiligree } [<options>]
```

A plain box decorated with the filigree frame, with a **FiligreeBackColor** background. It is equivalent to an untitled **RpgTip** (page 17) with the **filigree** option, and accepts the same options.

The filigree may also be added to any **tcolorbox** by including the **rpgfiligree** style in its options, or to a callout or table with the **filigree** option.

```

%% As environment
\begin{RpgFiligree}
  A basic filigree-wrapped tbox.
\end{RpgFiligree}

%% As option
\begin{RpgSidebar}[filigree]{Fancy Sidebar}
  A very fancy sidebar
\end{RpgSidebar}

```

A basic filigree-wrapped tbox.

Fancy Sidebar
A very fancy sidebar

Secrets

Game Masters often need notes which their players should not see. The `RpgSecret` environment allows the same document to serve both purposes, by marking text as secret and choosing whether to reveal it.

```
\begin{ RpgSecret } [<options>]{<public-text>}
```

Marks the body of the environment as secret. Its appearance depends on the `ShowSecrets` switch (page 15), which is off by default:

Switch	Display
Off	Only the (optional) <code>public-text</code> is shown, as ordinary text, with nothing to suggest that anything has been withheld.
On	The secret text is shown in a highlighted box, labelled as hidden. If <code>public-text</code> was given, it is shown in the lower part of the box, labelled as the publicly available version.

Secrets are revealed for the remainder of the document with `\RpgSwitch{ ShowSecrets }{ on }`. The following options are available:

Option	Effect
<code>color</code>	The colour of the box's frame. Default: <code>SecretColor</code> .
<code>label-public</code>	The label above the public text. Default: <code>`Normal text'</code> .
<code>label-secret</code>	The label above the secret text. Default: <code>`Hidden text'</code> .
<code>switch</code>	The switch which controls this secret. Default: <code>ShowSecrets</code> . Using a different switch allows separate groups of secrets to be revealed independently.

The labels are typeset in the `\RpgFontSecretTitle` (page 28) font element.

RpgSecret: Hidden

```

\RpgSwitch{ShowSecrets}{false}

\lorumIpsumOne%Snippets of standard lorem ipsum
\begin{RpgSecret}{}
  The players don't know it yet, but all this
  pseudo-latin text is a hint that devils are
  involved!
\end{RpgSecret}
\lorumIpsumTwo{}
\begin{RpgSecret}{
  Logos kykloforia, thelxis kai ethos, tyrannis
  kai epilysis. Aei theria plaka, kyrios meion
  egestas, anagkas proin eira kai theria.
}

  This bit of text is noteworthy - it's been
  added in after the fact
\end{RpgSecret}
\lorumIpsumThree

```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Logos kykloforia, thelxis kai ethos, tyrannis kai epilysis. Aei theria plaka, kyrios meion egestas, anagkas proin eira kai theria. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

```
%exactly the same as above, but reveal the secrets!
\RpgSwitch{ShowSecrets}{true}

\lorumIpsumOne%Snippets of standard lorem ipsum
\begin{RpgSecret}{
  The players don't know it yet, but all this
  pseudo-latin text is a hint that devils are
  involved!
}
\end{RpgSecret}
\lorumIpsumTwo{
\begin{RpgSecret}{
  Logos kykloforia, thelxis kai ethos, tyrannis
  kai epilysis. Aei theria plaka, kyrios meion
  egestas, anagkas proin eira kai theria.
}

  This bit of text is noteworthy - it's been
  added in after the fact
}
\end{RpgSecret}
\lorumIpsumThree
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

Hidden text

The players don't know it yet, but all this pseudo-latin text is a hint that devils are involved!

Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

Hidden text

This bit of text is noteworthy - it's been added in after the fact

Normal text

Logos kykloforia, thelxis kai ethos, tyrannis kai epilysis. Aei theria plaka, kyrios meion egestas, anagkas proin eira kai theria.

Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

Maps

Adventures often describe a location as a numbered list of areas, which correspond to the labels on a map. The `RpgMap` environment produces such a list, numbering and nesting the areas automatically.

```
\begin{ RpgMap } [<options>]
```

Begins a list of map areas, each of which is created with `\area`. Each area is given a heading, whose size depends on how deeply the map is nested: `\section*` at the top level, then `\subsection*`, and so on, down to `\subparagraph*`. These headings do not appear in the table of contents.

Areas are numbered according to their depth, cycling through numbers, capital letters and lower-case Roman numerals, so that the second area within the first area is 1B, and its third sub-area is 1B-iii. The following options are available:

Option	Effect
<code>empty-prefix</code>	Text placed before the number of an unnamed area. Default: <code>`Area '</code> .
<code>header-offset</code>	The number of levels by which to shrink the headings. Default: 0.
<code>prefix</code>	Text placed before the number of a named area. Default: <code>`Area '</code> .
<code>title</code>	A title for the map, typeset as a heading one level larger than its areas. It is only used by the outermost map.

```
\RpgMapRef { label }
```

`\RpgMapRef*` Typesets the name of the labelled area. The starred form typesets its number instead. This is a wrapper around `\ref` and `\nameref`.

```
\RpgMapRef{map:dais} is area
\RpgMapRef*{map:dais}, on
\RpgPage{map:dais}.
```

The Dais is area 1A, on page 22.

Inside a Map

Inside a map environment, the following commands and environments are valid.

`\area [<label>]{<name>}`

Adds an area to the current map, much as `\item` adds an item to a list. Its heading is ``<prefix><number>: <name>'`, or ``<empty-prefix><number>'` if no name is given. Any text which follows forms the description of the area.

If a label is given, the area may be referred to with `\RpgMapRef` or `\RpgPage`. `\area` is only defined within a map.

`\begin{ RpgArea } [<label>]{<name>}`

Adds an area to the current map, and begins a nested map within it, so that the body of the environment may contain sub-areas. It is equivalent to an `\area` followed by an `RpgMap` environment, and raises an error if used outside of a map.

Nested Map Areas

```
\begin{RpgMap}[header-offset=2]
  \begin{RpgArea}[map:hall]{Great Hall}
    A long and draughty hall.

    \area[map:dais]{The Dais} A raised
      stone platform.

    \area An unremarkable alcove.
  \end{RpgArea}

  \area[map:cellar]{Cellar} Damp and
    dark.
\end{RpgMap}
```

Area 1: Great Hall

A long and draughty hall.

Area 1A: The Dais. A raised stone platform.

Area 1B. An unremarkable alcove.

Area 2: Cellar

Damp and dark.

Cards

Cards are small, playing-card sized boxes, useful for handouts such as items, spells or abilities. If the content of a card is too long to fit, it is automatically continued onto further cards.

Cards are rarely used directly: they are most often created by the rule environments (page 24), which can display the same content either as normal text or as a card.

RpgCard example

```
\begin{RpgCard}[vmargin=0.1cm, hmargin=0.1cm,
  height=5.2cm, width=3.6cm]
  \raggedright\footnotesize This is an example
    card with lots of text\footnote{And maybe a
      footnote too?}.
  \lipsum[2]
\end{RpgCard}
```

This is an example card with lots of text*. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis.

Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

*And maybe a footnote too?

`\cardbreak {}`

Within an `RpgCard`, ends the current card and continues the text on the next, in the same way as `\pagebreak`. Outside of a card it has no effect, so it may safely be used in text which is only sometimes displayed as a card.

`\footnote { text }`

Within an `RpgCard`, `\footnote` is redefined so that its text appears at the end of the card's contents, below a horizontal rule. **Only one footnote may appear in each card:** if more than one is used, only the last is shown. Card footnotes do not affect the document's footnote numbering.

`\begin{ RpgCard } [ <options> ]`

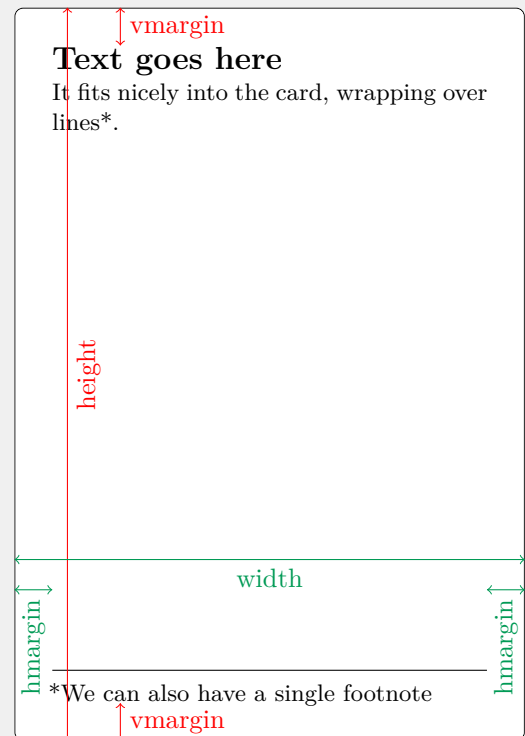
Typesets its contents in one or more cards, placed side by side. The text is typeset in the `\RpgFontCardBody` (page 28) font element. The following options are available, with the defaults given in brackets:

Option	Effect
<code>cardsep</code>	The horizontal space between consecutive cards (5pt).
<code>color, opacity</code>	The colour (<code>white</code>) and opacity (1) of the card's background.
<code>footnote</code>	The symbol used to mark the card's footnote (*).
<code>hmargin, vmargin</code>	The space between the edge of the card and its text (0.2cm). Each must be less than half of the corresponding dimension.
<code>vcentre</code>	If present, centres the text vertically within each card. (<code>vcenter</code> is also accepted.)
<code>width, height</code>	The outer dimensions of each card (6.75cm by 9.7cm).

Cards cannot be given a title, and cannot be nested within one another.

```
\begin{tikzpicture}%wrap in tikz so we can
  draw dimensions
  \node[anchor=south west,inner
    sep=0pt,outer sep=0pt] at (0,0)
  {
    \begin{RpgCard}[hmargin=0.5cm,
      vmargin=0.5cm, height=9.7cm,
      width=6.75cm]
      \subsubsection{Text goes here}
      It fits nicely into the card,
      wrapping over
      lines\footnote{We can also
        have a single footnote}.
    \end{RpgCard}\ignorespaces
  };
  %% Draw the dimenions on top; use macros
  to save space!
  \heightLines{red}
  \widthLines{ForestGreen}

\end{tikzpicture}
```



`\RpgCardReset {}`

Restores the default options of all subsequent cards to their original values.

`\RpgCardSet { options }`

Sets the default options of all subsequent cards. Options given to an individual `RpgCard` take priority over these defaults.

Rule Environments

The environments described so far are independent of any particular game system. Some elements, however, depend on the rules of the game being written for: a monster's statistics in one system look nothing like those in another. **RPGTeX** therefore provides four *rule environments*, whose properties and appearance are defined by the theme:

Environment	Purpose
RpgFeat	Feats, talents and other character features.
RpgItem	Items and equipment.
RpgSpell	Spells and similar abilities.
RpgStat	Statistics blocks for creatures and characters.

Each may be displayed either as normal text or as a card, controlled by a switch. Since the available properties depend on the theme, consult the documentation of the active theme for the details of each.

\RpgGetKey *(Variable)*

The value given to a property of the current rule environment, such as `\RpgGetKey{ rarity }`. It may only be used within a rule environment.

```
\begin{RpgItem}{<name>}[<key> = <value>]
  The value of the key is \RpgGetKey{<key>}.
\end{RpgItem}
```

```
\begin{ RpgIfEnv } { switch }[<options-on>]{ env-on }[<options-off>]{ env-off }
```

Begins the environment **env-on** (with **options-on**) if **switch** is on, and **env-off** (with **options-off**) otherwise. Any arguments which follow are passed to the chosen environment, so both environments must take the same arguments. This is the mechanism by which the rule environments switch between text and card modes, and it may be used to create similar switchable environments.

```
\begin{ Rpg[X] } [<card-options>]{ name }[<properties>]
```

Typesets the rule element 'X' called **name**, with the given **properties** (a list of key-value pairs defined by the theme). Its display depends on the corresponding switch (page 15):

Environment (X)	Switch
RpgFeat	UseFeatCard
RpgItem	UseItemCard
RpgSpell	UseSpellCard
RpgStat	UseStatCard

If the switch is on, the element is displayed as an **RpgCard** (page 23), to which the **card-options** are passed. Otherwise (the default), it is displayed as normal text, and the **card-options** are ignored.

PART II

rpgtex For Designers

Chapter 5: Fonts

RPGTeX allows a high degree of customisation of the fonts and typefaces used throughout a document. Rather than setting fonts directly, each part of the document (headings, table text, footers and so on) draws its font from a named *font element*, such as `\RpgFontSection` (page 28). The theme defines the appearance of each element using `\RpgThemeFont`, and assigns each element to the relevant part of the document. Changing the appearance of an element therefore changes it consistently wherever that element is used.

Why didn't my font change?

`\RpgThemeFont` does not change any fonts directly: it updates the stored definition of a *font element*, which a theme designer must then assign to the relevant part of the document. For example, the `\RpgFontSection` (page 28) element does not 'hook in' to the `\section` command by itself; it changes the section font only because most themes call `\titleformat{\section}{\RpgFontSection}...`. If the `\titleformat` were later changed (for whatever reason), the section font would no longer update.

If an element does not change font after its definition is updated, first check that the font element is actually being invoked -- and if not, invoke it manually. Once the command is in place, the font will change as expected.

Note also that `\RpgThemeFont` has no effect unless the `font` option is active (page 6). This is the default when using one of the RPGTeX classes.

Font Commands

`\RpgThemeFont { key-value-list }`

Updates the font elements specified by a comma-separated list of key-value pairs. Each element has two keys, `<name>-family` and `<name>-style`, where `<name>` is given in the 'Components' column of the table below (or was assigned by `\RpgThemeCreateFont`).

The values may be any L^AT_EX font or formatting commands (such as `\slshape\scriptsize\bfseries`), which are applied whenever the element is invoked. To change the typeface itself, a font family must first be defined, for example using `\newfontfamily` from the `fontspec` package.

Font Interface

```
\subsection{The Original Font}
Here is some text
```

```
\newfontfamily{\courier}{Courier}
\RpgThemeFont{main-body-family=\courier,
subsection-style=\slshape\Huge}
```

```
\subsection{The New Font}
And after the change is introduced
```

The Original Font

Here is some text

The New Font

And after the change is introduced

`\RpgThemeCreateFont { \ElementName }{ key-base }`

Creates a new font element, `\ElementName`, along with the keys `<key-base>-family` and `<key-base>-style`. A new element is initially blank (it applies no formatting) until it is assigned a value using `\RpgThemeFont`. Thereafter, it may be used and modified in exactly the same way as the built-in font elements.

Create Font

```
\RpgThemeCreateFont{\MyFont}{myfont}
\newfontfamily{\courier}{Courier}
\RpgThemeFont{
  myfont-family = \courier,
  myfont-style = \slshape
}

This is text

{\MyFont This is text in my new font}
```

This is text
This is text in my new font

Font Elements

RPGTeX provides 35 font elements by default (themes may define more). Since each element is assigned to part of the document by the theme designer, an element intended for one purpose (such as section headings) may be used for a different purpose within another theme. The descriptions below therefore reflect how the elements are used by RPGTeX itself and by the provided themes.

Family vs Style

Each font element is defined by two components: a **family** and a **style**. Formally, the **family** defines the **typeface** used by the element, whilst the **style** determines the options applied to that typeface (weight, shape, size, colour and so on).

In practice the distinction is largely one of clarity, since the font element simply applies the two in sequence:

```
\RpgFontX = <x-family> <x-style>
```

It is therefore generally safe to place commands which belong in the family within the style key, provided that they do not conflict with other styling.

Font vs Implementation

We encourage designers to place all text styling within the relevant font element, rather than elsewhere. If all subsections are to be red, then set `subsection-style=\color{ red }`, rather than setting the colour within the `titlesec` specification.

There will naturally be some exceptions to this: we found that the `RpgTitleFont` colour we wanted within `RpgDrawCover` diverged so strongly from that in `RpgSimpleTitle` that it made sense to define a special colour when rendering over a background image.

Font Element	Components	Usage
\RpgFontBody	main-body-family main-body-style	The main body text of the document. \RpgThemeFont sets \normalfont equal to this element. Changing the font size here (e.g. with \large) can give counterintuitive results, since it affects <i>only</i> the body text and does not scale the other elements accordingly. The font size of the whole document should instead be set using the <code>size</code> option (page 6).
\RpgFontEmphasis	emph-family emph-style	The font used by \RpgKey, and therefore by \emph unless the <code>noemph</code> option is passed.
\RpgFontTitle	title-family title-style	The font used for the document title, as returned by \RpgGetTitle and displayed by \maketitle.
\RpgFontSubTitle	subtitle-family subtitle-style	The font used for the subtitle and author, as returned by \RpgGetSubtitle and \RpgGetAuthor and displayed by \maketitle.
\RpgFontPart	part-family part-style	The font used for the part name on the part page created by \part.
\RpgFontChapter	chapter-family chapter-style	The font used for \chapter headings.

<code>\RpgFontSection</code>	section-family section-style	The font used for <code>\section</code> headings.
<code>\RpgFontSubsection</code>	subsection-family subsection-style	The font used for <code>\subsection</code> headings.
<code>\RpgFontSubsubsection</code>	subsubsection-family subsubsection-style	The font used for <code>\subsubsection</code> headings.
<code>\RpgFontParagraph</code>	paragraph-family paragraph-style	The font used for <code>\paragraph</code> headings.
<code>\RpgFontSubparagraph</code>	subparagraph-family subparagraph-style	The font used for <code>\subparagraph</code> headings.
<code>\RpgFontTableTitle</code>	table-title-family table-title-style	The title of an <code>RpgTable</code> (page 18), if one is set using the <code>title</code> option.
<code>\RpgFontTableHeader</code>	table-header-family table-header-style	The first (header) row of an <code>RpgTable</code> (page 18).
<code>\RpgFontTableBody</code>	table-body-family table-body-style	The rows of an <code>RpgTable</code> (page 18) after the header row.
<code>\RpgFontTipTitle</code>	tip-title-family tip-title-style	The title of an <code>RpgTip</code> (page 17) environment.
<code>\RpgFontTipBody</code>	tip-body-family tip-body-style	The body of an <code>RpgTip</code> (page 17) environment.
<code>\RpgFontSidebarTitle</code>	sidebar-title-family sidebar-title-style	The title of an <code>RpgSidebar</code> (page 17) environment.
<code>\RpgFontSidebarBody</code>	sidebar-body-family sidebar-body-style	The body of an <code>RpgSidebar</code> (page 17) environment.
<code>\RpgFontNarration</code>	narration-family narration-style	All text within an <code>RpgNarration</code> (page 17) environment (which has no title).
<code>\RpgFontAbstractTitle</code>	abstract-title-family abstract-title-style	The title of an <code>RpgAbstract</code> (page 19) environment.
<code>\RpgFontAbstractBody</code>	abstract-body-family abstract-body-style	The body of an <code>RpgAbstract</code> (page 19) environment.
<code>\RpgFontSecretTitle</code>	secret-title-family secret-title-style	The labels which distinguish the hidden and public text of an <code>RpgSecret</code> (page 20) environment when secrets are shown.
<code>\RpgFontCardTitle</code>	card-title-family card-title-style	The title of a card, as used by the card-switching environments of the provided themes (such as <code>RpgItem</code>).
<code>\RpgFontCardHeader</code>	card-header-family card-header-style	Subheadings and descriptive headers within a card (such as the type and rarity of an <code>RpgItem</code>).
<code>\RpgFontCardBody</code>	card-body-family card-body-style	The body text of an <code>RpgCard</code> (page 23).
<code>\RpgFontTocPart</code>	toc-part-family toc-part-style	Part entries in the table of contents.
<code>\RpgFontTocChapter</code>	toc-chapter-family toc-chapter-style	Chapter entries in the table of contents.
<code>\RpgFontTocSection</code>	toc-section-family toc-section-style	Section entries in the table of contents.
<code>\RpgFontStatBlockTitle</code>	stat-block-title-family stat-block-title-style	The title of a <code>statblock</code> environment (in the <code>dnd</code> theme, <code>RpgStat</code>).
<code>\RpgFontStatBlockBody</code>	stat-block-body-family stat-block-body-style	The body text of a <code>statblock</code> environment.
<code>\RpgFontStatBlockSection</code>	stat-block-section-family stat-block-section-style	Section headings within a <code>statblock</code> environment.
<code>\RpgFontFooter</code>	footer-family footer-style	The footer text.
<code>\RpgFontPageNumber</code>	page-number-family page-number-style	The page number within the footer.
<code>\RpgFontDropCap</code>	drop-cap-family drop-cap-style	The large initial letter created by <code>\RpgDropCap</code> .
<code>\RpgFontDropCapAfter</code>	drop-cap-after-family drop-cap-after-style	The text which follows the initial letter of an <code>\RpgDropCap</code> , on the first line.

Index

- abstract, 19
- area, 21, 22
- author, 10, 11
- background, 6
- bfseries, 13
- bg, 6
- cardbreak, 22
- centering, 15
- chapter, 12, 27
- chapter*, 12
- chaptermark, 10, 12
- colorlet, 8
- columns, 6
- cover, 10, 11
- definecolor, 8
- documentclass, 9
- edef, 2
- emph, 7, 14, 27
- eso-pic, 15
- font, 7
- footnote, 23
- Huge, 13
- item, 22
- justified, 7
- kpsewhich, 2
- large, 27
- linewidth, 15, 18
- maketitle, 8, 10, 11, 27
- nameref, 21
- newfontfamily, 26
- noemph, 7
- nomultitoc, 7
- normalfont, 27
- oneside, 7
- openright, 7
- pagebreak, 22
- paragraph, 28
- part, 10–12, 27
- part*, 10, 12
- ref, 21
- Rpg[X], 24
- RpgAbstract, 19, 28
- RpgArea, 22
- RpgCard, 23, 24, 28
 - Reset, 23
 - Set, 23
- RpgChapterName, 12
- RpgCMD, 4, 9
- RpgContour, 8, 13
- RpgCoverTransparency, 11, 15
- RpgDefaultPartImage, 11
- RpgDice, 13
- RpgDropCap, 14, 28
- RpgDropCapSeparation, 14
- RpgFake, 13
- RpgFeat, 24
- RpgFiligree, 8, 18, 19
- RpgFont
 - AbstractBody, 19, 28
 - AbstractTitle, 19, 28
 - Body, 27
 - CardBody, 23, 28
 - CardHeader, 28
 - CardTitle, 28
 - Chapter, 27
 - DropCap, 14, 28
 - DropCapAfter, 14, 28
 - Emphasis, 14, 27
 - Footer, 28
 - Narration, 17, 28
 - PageNumber, 28
 - Paragraph, 28
 - Part, 27
 - SecretTitle, 20, 28
 - Section, 26, 28
 - SidebarBody, 17, 28
 - SidebarTitle, 17, 28
 - StatBlockBody, 28
 - StatBlockSection, 28
 - StatBlockTitle, 28
 - Subparagraph, 28
 - Subsection, 28
 - Subsubsection, 28
 - SubTitle, 11, 27
 - TableBody, 18, 28
 - TableHeader, 18, 28
 - TableTitle, 18, 28
 - TipBody, 17, 28
 - TipTitle, 17, 28
 - Title, 11, 27
 - TocChapter, 28
 - TocPart, 28
 - TocSection, 28
- RpgGetAuthor, 11, 27
- RpgGetAuthor*, 11
- RpgGetKey, 24
- RpgGetName, 12
- RpgGetSubtitle, 11, 27
- RpgGetSubtitle*, 11
- RpgGetTitle, 11, 27
- RpgGetTitle*, 11
- RpgIfEnv, 24
- RpgIfSwitch, 16
- RpgItem, 24
- RpgKey, 7, 14, 27
- rpglatex, 3
- RpgMakeTitle, 10, 11
- RpgMap, 21
 - Ref, 21, 22
 - Ref*, 21
- RpgNarration, 8, 17, 28
- RpgNewSwitch, 16
- RpgOrdinal, 14, 15
- RpgPackagePath, 2
- RpgPage, 14, 22
- RpgPart
 - Name, 11, 12
 - Transparency, 12
- RpgPlural, 15
- RpgPut, 15
- RpgRowStyle, 18
- RpgSecret, 8, 20, 28
- RpgSectionName, 12
- RpgSetTheme, 7, 10
- RpgSetThemeColor, 8, 10
- RpgSetThemePath, 8, 10
- RpgSidebar, 8, 17, 28
- RpgSpell, 24
- RpgStat, 24
- RpgSubsectionName, 12
- RpgSubsubsectionName, 12
- RpgSuperOrdinal, 15
- RpgSwitch, 16, 20
- RpgTable, 8, 18, 28
- RpgTheme
 - CreateFont, 26
 - Font, 26, 27
 - FooterDecoration, 6
 - PaperColor, 6
 - PaperImage, 6
 - TitleMode, 11
- RpgTip, 8, 17, 19, 28
- RpgToggle, 16
- RpgWholePageImage, 15
- section, 12, 26, 28
- section*, 12, 21
- sectionmark, 10, 12
- sections, 7
- setlength, 14
- size, 7
- subparagraph, 28
- subparagraph*, 21
- subsection, 12, 28
- subsection*, 12, 21
- subsectionmark, 10, 12
- subsubsection, 12, 28
- subsubsection*, 12
- subsubsectionmark, 10, 12
- subtitle, 11
- summary, 19
- textbf, 14
- theme, 7
- themepath, 8
- title, 8, 10, 11
- usepackage, 9
- write18, 2